

From StableHLO to Tensix

libtt as a hermetic, full-stack runtime for Tenstorrent accelerators

libtt technical report

15 July 2026

Contents

Abstract	4
Executive summary	5
1 Design	7
1.1 Deployment model	7
1.2 Upstream-compatible frontend	7
1.3 One build graph	8
1.4 Cross-layer changes	8
2 Compilation and execution architecture	9
2.1 The framework boundary: JAX, StableHLO, and PJRT	9
2.2 TT-MLIR IR levels	9
2.3 TTNN runtime and the serialized executable	11
2.4 TT-Metal programs and Tensix kernels	11
2.5 Tiles, memory, and low-precision weights	11
2.6 Decode bottlenecks	12
3 Optimization map by patch and concept	13
4 Recovering model semantics in TTIR and TTNN	14
4.1 RMSNorm recognition	14
4.2 Expanded-SiLU recognition	14
4.3 Rank-3 RoPE and QKV projection structure	14
4.4 KV-cache dtype and role propagation	15
5 Layout, precision, and runtime policy	16
5.1 BF8 lowering and fast host packing	16
5.2 Width-sharded decode RMSNorm	16
5.3 Decode-specific validation changes	16
6 MLP optimizations	17
6.1 Shared-LHS gate/up projection	17
6.2 Consumer-side SiLU fusion	17
6.3 Dst-resident matmul-SwiGLU epilogue	17
6.4 Prefill coverage and fallback removal	18
6.5 SwiGLU K-blocking sweep	18
6.6 110-core fused-residual down projection	18
7 Runtime trace capture for short prefill	20
8 Packaging, build, and startup patch concepts	21
9 Benchmark methodology	22
9.1 Hardware and software	22
9.2 Serving command	22

9.3	Request and sample policy	23
9.4	Reported statistics	23
10	Results	24
10.1	Established concept sequence	24
10.2	Current kernel A/B experiments	26
10.3	Device profile of the down projection	26
10.4	Upstream tt-inference-server reference	26
10.5	Correctness and numeric behavior	28
11	Engineering rules	29
11.1	Match the abstraction to the information	29
11.2	Name the removed boundary	29
11.3	Down-projection geometry	29
11.4	Keep negative results	30
12	Limitations and follow-up work	31
13	Conclusion	32
	Bibliography	33

Abstract

libtt packages the Tenstorrent JAX stack as one PJRT shared library. Its goal is to run SGLang-JAX [21] and the broader JAX/TPU software ecosystem with few or no changes above the compiler boundary. It builds and pins TT-XLA [1], TT-MLIR [2], TTNN [3], TT-Metalium [4], device kernels, and runtime assets in `libtt.so`. The target needs compatible drivers and firmware, but no separate compiler or TT-Metal installation.

This approach has two advantages. libtt can stay close to upstream SGLang-JAX instead of reimplementing the model or inference engine, and it can recover performance below that boundary through StableHLO/MLIR transformations, runtime policy, and device kernels. The same boundary can support training in the future, although this report measures inference only. The report maps the changes to the TT-MLIR IR levels, runtime, and kernels where they are made.

The report follows that development sequence. A functional baseline first runs upstream SGLang-JAX at 16.265 tokens/s. Compiler, runtime, and kernel patches raise end-to-end throughput to 26.123 tokens/s (+60.61%). A 110-core down projection then reaches 27.753 decode tokens/s. Against a matched 32-sample tt-inference-server v0.10.0 run [25], current libtt is 11.51% faster in pure decode and 11.48% faster end to end.

Executive summary

`libtt.so` contains the TT-XLA PJRT implementation, TT-MLIR compiler, TTNN and TT-Metal runtime code, and a compressed archive of TT-Metal runtime assets. It extracts the archive on first load. This deployment model resembles Google’s `libtpu.so`, which contains the TPU compiler, driver, and hardware communication code [7].

One Bazel build pins the source revisions, applies the patch series, and builds the shared library. Its tools and dependencies are explicit inputs, following Bazel’s definition of a hermetic build [8]. A compiler rewrite, runtime change, and device kernel can therefore be built and benchmarked together.

The upper stack remains close to upstream SGLang-JAX and its JAX/TPU-oriented interfaces. This avoids a separate Tenstorrent model implementation and major changes to the scheduler or inference engine. Performance work instead happens in TT-MLIR, TTNN, TT-Metal, and the compiled graph.

The measurements follow the implementation sequence. V0 is a functional, relatively unoptimized backend that runs upstream SGLang-JAX. V1 through V9 add compiler, runtime, and kernel optimizations while keeping the upper stack largely unchanged. The final comparison uses the optimized backend against the TTIS reference stack.

The measured work supports six conclusions:

1. **Recover model semantics before lowering.** TTIR and TTNN patterns recover RMSNorm, SiLU, and QKV structure from primitive JAX operations. The foundation group adds 22.58%; SiLU recognition adds 3.30%; QKV fusion adds 1.15%.
2. **Use the runtime for shape-dependent layouts.** Width-sharding one-token RMSNorm across 16 cores adds 13.61%.
3. **Fuse at the producer boundary.** Combining gate/up matmuls alone changes throughput by -1.69%. Consumer-side SiLU fusion changes it by -0.65%. A matmul-SwiGLU epilogue that keeps both results in Dst adds 2.73%.
4. **Measure blocking choices.** Increasing the SwiGLU K block from two to four tiles changes throughput by +0.27%, so two remains the default.
5. **Use more cores without rereading weights.** The 110-core down projection raises effective BFP8 bandwidth from 246.2 to 345.2 GB/s, cuts kernel time by 28.7%, and adds 5.96% pure decode throughput.
6. **Trace short prefill.** Capturing the one-tile prefill graph reduces mean time to first token from 618.84 to 59.15 ms and improves the primary 128-token benchmark by 10.23%. Pure decode changes by +0.36%.

The current build reaches 27.753 decode tokens/s and 27.619 streaming end-to-end tokens/s. The matched TTIS reference, which uses TT-specific transformer and inference-engine layers, reaches 24.887 and 24.776 tokens/s, respectively.

libtt: one PJRT artifact from framework IR to Tensix execution

UPSTREAM FRONTEND · OUTSIDE LIBTT · LARGELY UNCHANGED

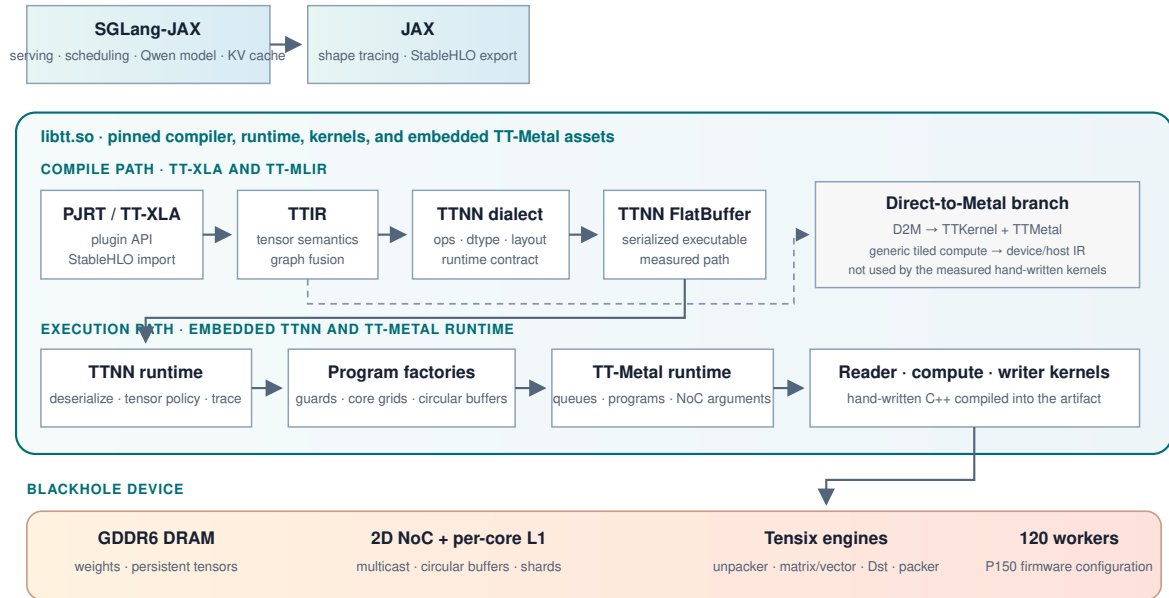


Figure 1: libtt’s serving, compilation, runtime, and device layers.

1 Design

1.1 Deployment model

PJRT lets frameworks call device-specific plugins through one interface [9]. libtt implements that interface for Tenstorrent:

JAX process

```
`-- dynamically loads libtt.so through PJRT
    |-- compiles StableHLO for Tenstorrent
    |-- allocates and transfers device buffers
    |-- loads and executes serialized TTNN programs
    `-- initializes the embedded TT-Metal runtime assets
```

libtt's C++ glue extracts the embedded archive to a fingerprinted temporary directory, sets `TT_METAL_RUNTIME_ROOT` before PJRT starts, links the upstream plugin, and exports only the PJRT entry points. Most code comes from pinned open-source dependencies and the applied patch series.

The compiler and TT-Metal runtime do not need to be installed on the target. With compatible drivers and firmware, `libtt.so` contains the software needed to compile and run JAX programs. This is the intended similarity to `libtpu.so` [7].

1.2 Upstream-compatible frontend

libtt treats PJRT and StableHLO as the integration boundary. SGLang-JAX keeps its model, tokenizer, scheduler, paged KV cache, and inference engine. The Tenstorrent-specific work stays below that boundary instead of requiring a forked Qwen implementation or a rewritten serving layer.

This provides two advantages:

- upstream SGLang-JAX and JAX/TPU code can be reused with small configuration changes; and
- performance can be added in TT-MLIR, TTNN, TT-Metal, and device kernels, where layout, fusion, precision, and data movement are visible.

The report evaluates inference, but PJRT and StableHLO are not inference-specific. Training support can use the same architecture by adding the required operations, collective behavior, optimizer graphs, and numerical validation below the existing JAX frontend.

1.3 One build graph

libtt uses Bazel modules and repository rules to pin TT-UMD [5], SFPI [6], TT-Metal, LLVM, StableHLO, TT-XLA, TT-MLIR, Shardy, and supporting C++ libraries. Bazel overlays add targets for projects that use another build system. The `//:tt` target produces `bazel-bin/libtt.so`.

This organization has three practical effects:

- Source revisions, overlays, patches, toolchains, and link rules are inputs to one build.
- Compiler, schema, runtime, program-factory, and kernel changes can be tested together.
- An agent can inspect and edit every layer in one workspace, rebuild one target, and run one benchmark.

Tenstorrent’s compiler overview describes the same open path from framework frontends through TT-MLIR dialects to TT-Metalium [12, 13]. libtt builds a pinned part of that path as one artifact.

1.4 Cross-layer changes

The matmul-SwiGLU epilogue crosses six layers:

1. A graph pass must prove that a paired gate/up matmul feeds the exact `silu(gate) * up` expression.
2. TTNN IR needs an operation-level representation that preserves the fused semantic through lowering.
3. The serialized executable needs to carry the new matmul attribute.
4. The runtime must dispatch that operation into TTNN.
5. TTNN must select a program factory only for supported shapes, dtypes, layouts, and hardware.
6. Reader, compute, and writer kernels must keep the paired tiles resident in Dst and emit only the half-width final result.

These steps form one patch concept even though they touch several abstraction levels and two upstream repositories.

2 Compilation and execution architecture

2.1 The framework boundary: JAX, StableHLO, and PJRT

Upstream SGLang-JAX owns serving, tokenization, scheduling, the JAX Qwen model, and the paged KV cache [21]. JAX traces each shape-specific computation to StableHLO, a portable operation set between frameworks and compilers [10]. PJRT hides the device-specific compiler and runtime from JAX [9].

The steady-state request path is:

```
SGLang-JAX → JAX trace → StableHLO → libtt/PJRT
              → TT-XLA → TT-MLIR → TTNN executable
              → TTNN runtime → TT-Metal programs → Blackhole
```

`libtt.so` accepts StableHLO and executes a serialized TTNN program.

2.2 TT-MLIR IR levels

MLIR supports dialects at different abstraction levels [11]. TT-MLIR uses them for model semantics, layouts, tiled computation, device kernels, and host/device control [13]. Each optimization should use the highest level that still has the information it needs.

Table 2.1: The TT-MLIR IR ladder and where libtt’s measured path uses it.

Representation	Abstraction and purpose	Relationship to current libtt optimizations
StableHLO	Framework-portable tensor graph with specified operation semantics [10].	Input to TT-XLA/TT-MLIR. Framework algebra such as expanded SiLU is visible here and after import, but libtt’s patches generally match it in TTIR/TTNN passes.
TTIR	Hardware-aware, high-level tensor IR. It preserves named tensor operations while permitting fusion and canonicalization before a concrete runtime API is chosen [13].	JAX RMSNorm recognition, expanded-SiLU recovery, shared-LHS matmul grouping, role propagation, and part of QKV/RoPE fusion.

Representation	Abstraction and purpose	Relationship to current libtt optimizations
TTNN dialect	High-level tensor IR designed to model the TTNN library closely. Its types and attributes carry tiled layouts, memory spaces, sharding, and device data types [13, 14].	QKV composite fusion, <code>matmul_swiglu</code> , layout selection, cache return types, and lowering to the serialized TTNN operation graph.
TTNN FlatBuffer	Serialized executable operation graph consumed by the TTNN runtime. It is an artifact rather than an MLIR dialect.	Carries operation parameters such as the fused-SwiGLU <code>matmul</code> flag across the compiler/runtime boundary.
D2M dialect	Generic tensor/memref computation analogous to <code>linalg.generic</code> , augmented with grids, sharded tensors, circular buffers, and explicit data movement [13].	An alternative direct-to-metal route. The measured custom SwiGLU and down-projection kernels are hand-written TT-Metal/TTNN patches, so they do not pass through D2M.
TTKernel dialect	Low-level device-kernel IR exposing circular buffers, tile registers, NoC transactions, and synchronization with an intended near one-to-one mapping to TT-Metal kernels [13].	Relevant to generated direct-to-metal kernels; not the representation of the hand-written C++ kernels measured here.
TTMetal dialect	Host/device interop IR for allocation, transfers, program creation, and enqueue operations [13].	Part of the direct-to-metal compiler route. The measured TTNN route instead invokes TT-Metal host APIs from the TTNN runtime.

The main lowering branch in this report is therefore:

```

StableHLO → TTIR → TTNN dialect → TTNN FlatBuffer
                        ↓
                    TTNN C++ runtime
                        ↓
                TT-Metal host + C++ kernels

```

TT-MLIR also supports the lower-level branch:

```

TTIR → D2M → TTKernel + TTMetal → generated device/host program

```

Only the first branch runs the benchmarked patches. Their hand-written C++ program factories and kernels are not TTKernel IR optimizations.

2.3 TTNN runtime and the serialized executable

TTNN IR models operations such as matmul, RMSNorm, SDPA, and reshape. Lowering serializes them into a FlatBuffer. The runtime deserializes the graph, creates tensors, sets memory configurations, validates each operation, and selects a program factory. The RMSNorm patch uses the concrete input shape here to choose a 16-core width-sharded layout; the compiler still emits a normal RMSNorm.

2.4 TT-Metal programs and Tensix kernels

TT-Metalium uses cooperative dataflow rather than a GPU thread hierarchy [15]. A program usually has:

- a reader data-movement kernel to fetch tiles from DRAM or another core into L1 circular buffers;
- a compute kernel to unpack tiles, drive matrix/vector engines, and create result tiles; and
- a writer data-movement kernel to drain result buffers to their destination.

Circular buffers are bounded queues shared by the threads of a Tensix core [17]. Reader, compute, and writer kernels can overlap on different tiles when their buffer and semaphore protocols are correct.

The compute engine exposes SrcA, SrcB, and Dst register sets. Matrix results land in Dst; vector operations can transform them; the packer writes them to an L1 circular buffer [16]. With 16-bit Dst storage and double buffering enabled, the active half contains eight 32-by-32 tiles. That capacity determines the successful SwiGLU geometry: four gate and four up tiles fill Dst exactly.

2.5 Tiles, memory, and low-precision weights

TTNN’s standard tile is 32 by 32 elements with four 16-by-16 faces [18]. Final dimensions are padded, so batch-one decode still presents a full tile row. Kernels must account for both the logical and padded shapes.

Large weights reside in device DRAM; L1 is smaller, faster, and private to each worker. TTNN **MemoryConfig** values describe interleaved or sharded storage and DRAM or L1 placement. TT-MLIR layout attributes similarly encode how a logical tensor maps to devices, cores, physical shards, memory space, and padding [14].

The benchmark uses BF16 activations and outputs with BFLOAT8_B weights. BFLOAT8_B is block floating point: 16 values share an exponent [19]. It reduces the dominant weight traffic in batch-one decode, with a precision and packing trade-off. The custom matmuls use BF16 packer-L1 accumulation: partial sums are packed into an L1 slot and reloaded for later K blocks, leaving Dst available for the current block’s tile group.

2.6 Decode bottlenecks

Qwen3 is a decoder-only Transformer family [22]. A simplified layer is:

$$\begin{aligned} u &= \text{RMSNorm}(h), \\ h' &= h + \text{Attention}(Q(u), K(u), V(u); K_{\text{cache}}, V_{\text{cache}}), \\ v &= \text{RMSNorm}(h'), \\ g &= vW_{\text{gate}}, \quad r = vW_{\text{up}}, \\ h_{\text{next}} &= h' + (\text{SiLU}(g) \odot r)W_{\text{down}}. \end{aligned}$$

RMSNorm and SwiGLU are standard model concepts [23, 24]. Prefill shares each weight read across many rows. Serial decode applies a logical $1 \times K$ by $K \times N$ projection for every token, so weight traffic and intermediate materialization dominate. The main optimizations add width parallelism, reduce DRAM traffic, or avoid packing an intermediate from Dst.

3 Optimization map by patch and concept

Git revisions remain in the data files. The table groups changes by patch concept and implementation level.

Table 3.1: Current model-path optimization concepts, implementation levels, and measured effects.

Concept	Principal level / IR	Measured effect
Recover JAX RMSNorm	TTIR graph rewrite	Included in +22.58% foundation bundle
Recognize expanded SiLU	TTIR graph rewrite	+3.30%
Fuse rank-3 decode RoPE	TTIR/TTNN composite resolution	Included in foundation bundle
Fuse Qwen decode QKV structure	TTIR ordering + TTNN fusion	+1.15%
Preserve KV-cache result types	TTNN type inference	Included in foundation bundle
Enable BF8 activation/weight path	Compile options, TTNN lowering, host packing	Foundation/baseline; not isolated
Permit decode layouts	TTNN/TT-Metal validation	Included in foundation bundle
Width-shard decode RMSNorm	TTNN runtime policy	+13.61%
Combine two shared-LHS matmuls	TTIR fusion	-1.69%; enables epilogue
True matmul-SwiGLU epilogue	TTNN IR/FlatBuffer/runtime + TT-Metal	+2.73%
Generalize and remove fallback	TTNN matching/runtime simplification	-0.19%
Sweep SwiGLU K blocking	TT-Metal program and CB geometry	+0.27%
Specialize down projection	TTNN program selection + TT-Metal	+5.96% pure decode
Trace one-tile prefill	TTNN/TT-Metal runtime trace	-90.44% TTFT; +10.23% E2E

Patch index:

RMSNorm graph	tt_mlir_fuse_jax_rms_norm.patch
expanded SiLU	tt_mlir_fuse_expanded_silu.patch
decode RoPE	tt_mlir_fuse_rank3_rope_decode.patch
QKV structure	tt_mlir_qwen_decode_qkv_projection_fusion.patch
KV-cache types	tt_mlir_kv_cache_dtype_return_types.patch
BF8 path	tt_mlir_single_chip_activation_dtype_lowering.patch
	fast_bfloat16_bfp8_pack.patch
decode layouts	sdpa_decode_allow_l1_interleaved_q.patch
	layernorm_allow_single_core_height_sharded.patch
RMSNorm sharding	tt_mlir_sharded_decode_rms_norm.patch
shared-LHS pair	tt_mlir_fuse_shared_lhs_matmul_pairs.patch
SwiGLU vertical	tt_mlir_fuse_matmul_swiglu.patch
	matmul_swiglu_epilogue.patch
down projection	down_projection_110_core.patch

The +22.58% foundation figure combines patches that were not measured separately. The blocking and down-projection tests use a streaming decode clock, so they remain outside the cumulative sequence.

4 Recovering model semantics in TTIR and TTNN

4.1 RMSNorm recognition

JAX can lower RMSNorm to square, reduce, epsilon addition, reciprocal square root, scaling, and reshapes. A TTIR pattern proves this structure and replaces it with one RMSNorm operation while the algebra and use-def graph are still available. TTNN can then apply its normalization and layout logic. Recognition is part of the foundation bundle and enables the later runtime-sharding patch.

4.2 Expanded-SiLU recognition

The graph represents

$$\text{SiLU}(x) = x \sigma(x) = \frac{x}{1 + \exp(-x)}$$

with casts, broadcasts, reshapes, and a splatted scalar one. `tt_mlir_fuse_expanded_silu.patch` looks through view-like operations, verifies the constant and shared input, and replaces the tree with SiLU. The isolated gain is **+3.30%**.

4.3 Rank-3 RoPE and QKV projection structure

Qwen projects Q, K, and V together, applies per-head RMSNorm to Q and K, reshapes them by head count, and applies rotary position embedding. Two patches recover this structure:

- `tt_mlir_fuse_rank3_rope_decode.patch` accepts JAX’s rank-3 decode form, temporarily maps it to the existing rank-4 composite, and restores the expected result shape.
- `tt_mlir_qwen_decode_qkv_projection_fusion.patch` orders projection roles, validates contiguous Q/K/V bounds, head counts, and head dimensions, and replaces materialized slices/reshapes with `nlp_create_qkv_heads_decode`. Q and K RMSNorm are recreated on the fused rank-4 results.

TTIR supplies candidate ordering and role information; TTNN owns the composite operation and runtime contract. QKV fusion adds **+1.15%**. Rank-3 RoPE is part of the foundation bundle.

4.4 KV-cache dtype and role propagation

KV-cache update operations are stateful boundaries. Their return types must carry the selected device dtype, and graph-role metadata must survive unary operations so later fusions can still identify query, key, and value paths. The TT-MLIR and TT-XLA patches keep the low-precision fused graph typed and recognizable. They are enabling changes rather than isolated speedups.

5 Layout, precision, and runtime policy

5.1 BF8 lowering and fast host packing

The server requests `bfp_bf8` for eligible weights. TT-XLA enables single-chip lowering, TT-MLIR assigns the device dtype, and TTNN/TT-Metal consume BFLOAT8_B tiles. `fast_bfloat16_bfp8_pack.patch` speeds conversion of BF16 host weights to 32-by-32 BFP8 tiles.

These changes attack two different phases:

- BF8 storage reduces steady-state device weight traffic during decode.
- faster packing reduces model-load and preparation cost on the host.

The measurements do not isolate these effects from the foundation bundle.

5.2 Width-sharded decode RMSNorm

The default logical height of RMSNorm during batch-one decode is one. A generic height-parallel program cannot occupy many cores. The runtime patch recognizes a device-resident, tiled, unsharded input with sequence length one and width at least 2048. When the width divides into 16 tile-aligned shards, it:

1. creates an 8-by-2 row-major core grid;
2. width-shards the tensor into the cores' L1 memories;
3. derives the layernorm program configuration from that shard specification;
4. executes sharded RMSNorm; and
5. restores the requested output memory configuration.

The two conversions cost less than the width-parallel reduction saves. The isolated gain is **+13.61%**. This is runtime policy, not a new IR operation: the compiler identifies RMSNorm, while the runtime knows the tensor and device geometry.

5.3 Decode-specific validation changes

Two small TT-Metal patches admit layouts selected by the optimized graph:

- SDPA decode accepts an L1-interleaved query rather than requiring the prior sharded form.
- layernorm accepts the single-core height-sharded case produced on the decode path.

These narrow changes remove conversions while keeping shape and layout checks. Their performance effect is part of the foundation bundle.

6 MLP optimizations

6.1 Shared-LHS gate/up projection

Qwen’s two first MLP projections share an activation:

$$xW_{gate}, xW_{up} \longrightarrow x[W_{gate} \ W_{up}].$$

`tt_mlir_fuse_shared_lhs_matmul_pairs.patch` changes TTIR fusion eligibility so a pair can be combined. This removes duplicate activation reads and launches, but writes a doubled-width output that SwiGLU slices and rereads. Throughput changes by **-1.69%**. The representation becomes useful when the epilogue consumes it without materializing the combined output.

6.2 Consumer-side SiLU fusion

An intermediate experiment placed SiLU as a unary activation on the following TTNN multiply. That removes one elementwise operation but retains the expensive boundary:

```
combined matmul → pack full gate/up tensor → memory  
                → read/unpack → SiLU + multiply → pack result
```

Throughput changes by **-0.65%**. Removing an operation does not help when the same large intermediate is still written and read.

6.3 Dst-resident matmul-SwiGLU epilogue

Two patches implement the producer-side epilogue:

- `tt_mlir_fuse_matmul_swiglu.patch` recognizes the TTNN graph, introduces the fused matmul semantic, extends the FlatBuffer representation, serializes it, and invokes the corresponding runtime path.
- `matmul_swiglu_epilogue.patch` adds TTNN validation, a program factory, and the TT-Metal reader/receiver/sender/compute kernels.

The contract supports one physical tile row, BF16 activation/output, BFLOAT8_B weights, DRAM-interleaved tensors, no transpose or bias, and a grid that can supply 96 workers. For Qwen3-8B, each worker owns four gate tiles and four corresponding up tiles—exactly the eight active 16-bit Dst slots.

The program performs the following sequence:

1. two sender cores multicast activation K blocks over the NoC;
2. each of 96 workers reads its paired gate/up weight tiles;

3. the matrix engine accumulates all eight output tiles;
4. partial K results use BF16 packer-L1 accumulation;
5. final gate/up partials are loaded together into Dst;
6. SiLU transforms Dst slots 0–3;
7. each gate tile multiplies its corresponding up tile in place; and
8. only four final BF16 tiles are packed and written.

The doubled-width gate/up tensor is never materialized. End-to-end throughput improves by **+2.73%**.

6.4 Prefill coverage and fallback removal

The matcher was then generalized from decode-specific naming to any supported one-tile-row input, which includes short prefill. The older consumer-side SiLU-multiply fallback was removed; validation remains at the TTNN/TT-Metal boundary, where unsupported dtype, layout, shape, and hardware combinations fail explicitly.

Throughput changes by **-0.19%**. The change removes code and adds prefill coverage without changing the fast kernel or completion hash.

6.5 SwiGLU K-blocking sweep

The initial epilogue uses `in0_block_w = 2`. Every two K tiles it packs eight partial tiles into L1 and later reloads them. Larger blocks reduce partial traffic but need larger activation and weight circular buffers.

`matmul_swiglu_epilogue.patch` now exposes `TT_METAL_SWIGLU_IN0_BLOCK_W={2,4,8,16}` and sizes the circular buffers from the chosen value. The 32-sample test compares width 4 with width 2 while disabling the new down-projection specialization:

K block	Pure decode mean $\pm$ SD	95% CI	Change
2 tiles	26.122 $\pm$ 0.402 tok/s	[25.977, 26.267]	—
4 tiles	26.192 $\pm$ 0.338 tok/s	[26.071, 26.314]	+0.27%

The intervals overlap and the measured change is small, so width two remains the default. Widths 8 and 16 remain available for experiments. Wider blocks also change accumulation order and can change greedy token sequences.

6.6 110-core fused-residual down projection

The second MLP matmul has the exact decode shape 1×12288 by 12288×4096 . The generic program used 64 cores and achieved only 246.2 GB/s of effective BFP8 weight bandwidth, well below the SwiGLU kernel. `down_projection_110_core.patch` adds an exact-shape Blackhole program selected only for BF16 activation/output, BFLOAT8_B weights, a BF16 residual, DRAM-interleaved tensors, and an 11-by-10 worker grid.

The program avoids a K split and cross-core reduction. It partitions the 128 output tile columns across 110 workers:

- 18 wide workers each compute two N tiles, covering columns 0–35;
- 92 narrow workers each compute one N tile, covering columns 36–127;
- one activation sender multicasts to the other 109 workers;
- weights are read exactly once;
- K is processed four tiles at a time with packer-L1 accumulation; and
- the residual addition occurs in the final compute kernel before output.

Wide and narrow workers need different partial-result circular buffers. An early version gave narrow workers a two-tile ring even though they produced one tile. Partial sums alternated slots and corrupted output. Separate one- and two-tile rings fixed the bug.

The exact-shape specialization is enabled by default. `TT_METAL_DOWN_PROJECTION_110_CORES=0` selects the generic program in the same binary for A/B measurement.

7 Runtime trace capture for short prefill

Decode replays a recorded TT-Metal command sequence while refreshing request-dependent buffers. Previously, the five-token prompt, padded to one 32-row tile, ran as separate host submissions.

Setting `SGLANG_TT_TRACE_DECODE_ONLY=false` records the fixed-shape prefill sequence too. In a same-binary 32-sample A/B test:

Metric	Decode-only trace	Prefill + decode trace	Change
Time to first token	618.84 $\pm$ 4.76 ms	59.15 $\pm$ 3.70 ms	-90.44% (10.46x)
Total streaming time	5.5067 $\pm$ 0.0598 s	4.9299 $\pm$ 0.0680 s	-10.48%
Pure decode throughput	25.986 $\pm$ 0.315 tok/s	26.079 $\pm$ 0.362 tok/s	+0.36%

Tracing removes about 560 ms before the first token while pure decode changes by +0.36%. In the primary cumulative benchmark, it adds **+10.23%** end-to-end throughput for a 128-token response.

8 Packaging, build, and startup patch concepts

Other patches make the single-library deployment buildable and keep startup cost manageable. They are not decode optimizations.

Table 8.1: Non-model patch groups in the current libtt build.

Concept	Representative files	Purpose
Embedded runtime assets	<code>tt_metal_runtime_root.cc</code> , <code>tt_metal_runtime_auto_setup.cc</code> , Bazel embed rules	Compress TT-Metal runtime files into <code>libtt.so</code> , extract to a fingerprinted temporary root, and initialize before PJRT.
Static integration and symbol hygiene	<code>BUILD.bazel</code> , <code>libtt_pjrt_exports.lds</code>	Link the upstream plugin and internal libraries into one shared object while exposing only PJRT ABI symbols.
Bazel overlays	<code>third_party/*/BUILD.overlay.bazel</code> , <code>tt_xla.BUILD.bazel</code> , <code>tt_mlir.BUILD.bazel</code>	Bring, separately built upstream projects into one central graph with pinned sources and toolchains.
Remove unused fabric/runtime surfaces	<code>disable_fabric_*.patch</code> , <code>disable_inspector_runtime.patch</code>	Exclude components unnecessary for the single-chip libtt artifact and avoid unresolved or duplicated runtime dependencies.
Linkability and TLS constraints	<code>reduce_static_tls.patch</code> , <code>remove_initial_exec_tls.patch</code>	Make large statically integrated components safe to load as a PJRT shared library.
Compiler/runtime trimming	public-UMD include patch TTNN-only registration/runtime patches, cold stubs, disabled TT-Lang resolver, fast sharded-module check	Avoid bringing unused compiler/runtime paths into the artifact and reduce cold compilation work.
API/version compatibility	protobuf, Shardy, affine, constructor-name, map/include patches	Reconcile pinned revisions inside one build without relying on a preinstalled matching stack.

These patches keep the integrated build loadable and remove unused subsystems.

9 Benchmark methodology

9.1 Hardware and software

Item	Value
Accelerator	Tenstorrent Blackhole P150
Firmware observed at startup	19.6.0
Host CPU	Intel Core Ultra 9 185H, 22 logical CPUs online
Host OS/kernel	Linux 6.8.0-124-generic, x86-64
Model	Qwen/Qwen3-8B
Model/activation dtype	BF16
Weight-lowering request	bfp_bf8
Attention backend	TT
Serving frontend	/home/pcmoritz/sglang-jax at 24eb823ed97e
Established main snapshot	627a32d
Current kernel snapshot	37d5460
Benchmark date	15 July 2026, America/Los_Angeles

The P150 exposes 120 Tensix workers in this firmware configuration [20]. The SwiGLU program uses 96; the new down projection uses 110. Results are single-device and shape-specific.

9.2 Serving command

The server uses the command documented in the repository README:

```
env -u TT_METAL_RUNTIME_ROOT \
  PYTHONPATH=/home/pcmoritz/sglang-jax/python \
  PJRT_NAMES_AND_LIBRARY_PATHS=tt:/home/pcmoritz/libtt/bazel-bin/libtt.so \
  JAX_PLATFORMS=tt \
  JAX_USE_SHARDY_PARTITIONER=false \
  JAX_COMPILATION_CACHE_DIR=/tmp/sglang-jax-qwen3-8b-jax-cache \
  SGLANG_TT_HOST_WEIGHT_LOAD=1 \
  SGLANG_TT_OPTIMIZATION_LEVEL=1 \
  SGLANG_TT_EXPERIMENTAL_WEIGHT_DTYPE=bfp_bf8 \
  SGLANG_TT_TRACE_DECODE_ONLY=false \
  /home/pcmoritz/sglang-jax/.venv/bin/python -m sgl_jax.launch_server \
  --model-path Qwen/Qwen3-8B \
  --host 127.0.0.1 --port 31000 \
  --device tt --dtype bfloat16 --attention-backend tt \
  --max-running-requests 2 --max-total-tokens 1024 \
  --max-prefill-tokens 256 --chunked-prefill-size 256 \
  --page-size 32 --watchdog-timeout 1200 \
```

```
--disable-precompile --skip-server-warmup \  
--disable-overlap-schedule --disable-radix-cache
```

Requests are serial despite the capacity of two. Radix caching and overlap scheduling are disabled so every request executes the same prompt prefill and does not overlap another request. Optional CPU/fused sampling paths remain at their disabled defaults; this report concerns the main model path.

9.3 Request and sample policy

The primary sequence sends the same request 34 times:

```
{  
  "text": "The capital of France is",  
  "sampling_params": {"temperature": 0, "max_new_tokens": 128}  
}
```

The first two requests are discarded because they include graph compilation, TT-Metal kernel compilation, trace setup, and cache population. The next 32 complete requests form the analysis window. Primary throughput is

$$T_j = \frac{128}{L_j},$$

where L_j is SGLang’s server-reported end-to-end latency.

The current kernel experiment uses the streaming completions endpoint and 32 retained requests per configuration. Pure decode throughput is 127 inter-token intervals divided by the elapsed time from first to last token. Streaming end-to-end throughput is 128 divided by loopback-client total time. The exact per-request observations are in `data/current-kernel-samples.csv`.

9.4 Reported statistics

Tables report the mean, sample standard deviation, and two-sided 95% Student-t interval. These describe variation within each recorded window but do not remove sequential drift, autocorrelation, or model and shape dependence.

10 Results

10.1 Established concept sequence

V0 is the functional baseline: it runs upstream SGLang-JAX but has few model-specific performance optimizations. V1 through V9 add compiler, runtime, and kernel changes below the JAX/PJRT boundary. Commit IDs are recorded in the CSV files.

Table 10.1: End-to-end 128-token generation, 32 retained requests per measurement stage.

Stage	Concept introduced	Mean $\pm$ SD (tok/s)	95% CI	Incremental	vs. baseline
V0	Documented serving baseline	16.265 $\pm$ 0.128	[16.218, 16.311]	—	0.00%
V1	Decode foundation: semantic recovery, dtype, and layout	19.938 $\pm$ 0.283	[19.836, 20.040]	+22.58%	+22.58%
V2	Expanded-SiLU recognition	20.596 $\pm$ 0.279	[20.495, 20.696]	+3.30%	+26.63%
V3	QKV projection fusion	20.832 $\pm$ 0.293	[20.726, 20.938]	+1.15%	+28.08%
V4	Two-way shared-LHS matmul	20.479 $\pm$ 0.321	[20.363, 20.594]	-1.69%	+25.91%
V5	Decode RMSNorm width sharding	23.265 $\pm$ 0.235	[23.181, 23.350]	+13.61%	+43.04%
V6	Consumer-side SiLU/multiply fusion	23.113 $\pm$ 0.296	[23.007, 23.220]	-0.65%	+42.11%
V7	Dst-resident matmul-SwiGLU epilogue	23.744 $\pm$ 0.311	[23.632, 23.856]	+2.73%	+45.98%
V8	Prefill-capable, fallback-free path	23.698 $\pm$ 0.274	[23.599, 23.797]	-0.19%	+45.70%
V9	Fixed-shape prefill trace	26.123 $\pm$ 0.394	[25.981, 26.265]	+10.23%	+60.61%

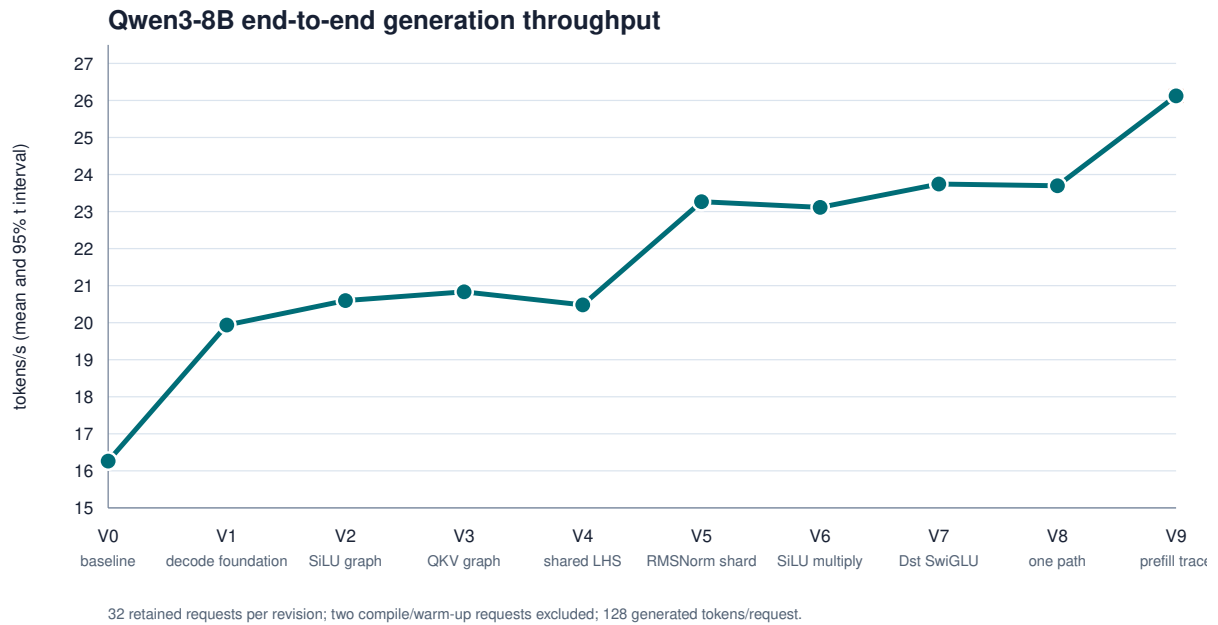


Figure 10.1: Cumulative throughput across the measured concepts.

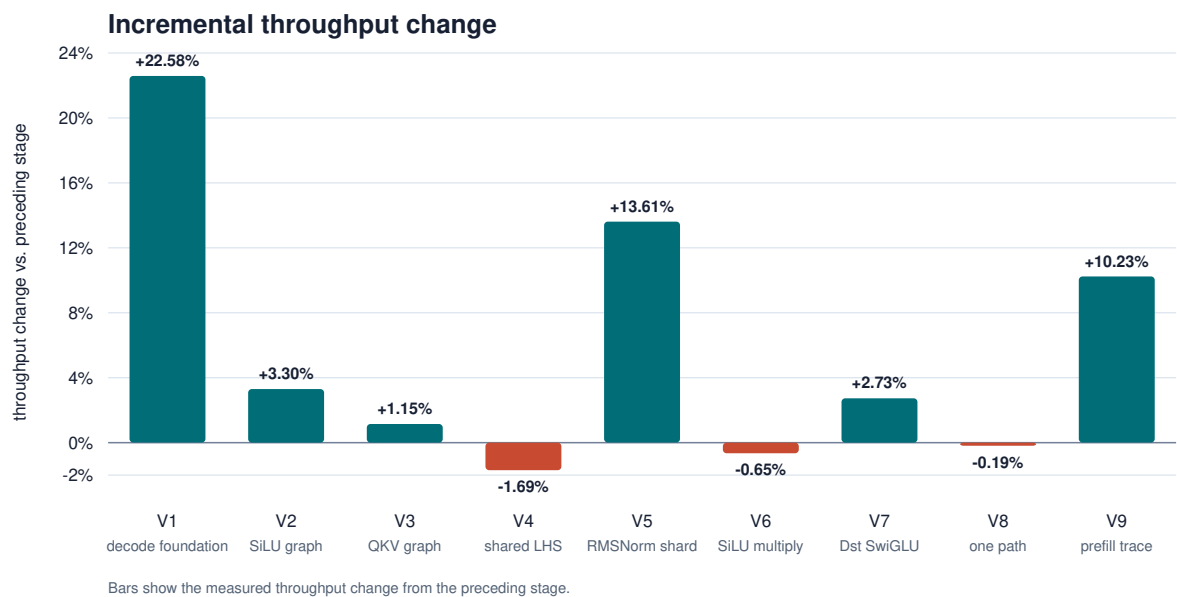


Figure 10.2: Incremental effect of each concept in the established sequence.

The negative and near-zero rows explain the final design: shared-LHS fusion materializes too much data, consumer-side SiLU leaves the producer boundary, and fallback removal changes coverage rather than the kernel.

10.2 Current kernel A/B experiments

Table 10.2: Current-branch streaming results, 32 retained samples per configuration. Width 4 is compared with width 2; the down projection is compared with the same width-4 binary/configuration with the specialization disabled.

Configuration	Pure decode mean $\pm$ SD	95% CI	Streaming E2E mean $\pm$ SD	Change in decode
SwiGLU K block 2, generic down	26.122 $\pm$ 0.402	[25.977, 26.267]	26.016 $\pm$ 0.396	—
SwiGLU K block 4, generic down	26.192 $\pm$ 0.338	[26.071, 26.314]	26.085 $\pm$ 0.333	+0.27%
K block 4, 110-core down	27.753 $\pm$ 0.330	[27.634, 27.872]	27.619 $\pm$ 0.322	+5.96%

The down projection cuts mean decode time from 38.179 to 36.032 ms/token, a 2.147 ms/token saving. Streaming end-to-end throughput rises by 5.88%, from 26.085 to 27.619 tokens/s. TTFT remains about 58 ms.

10.3 Device profile of the down projection

The direct TT-Metal profile identifies 36 occurrences per token, matching the 36 Qwen3-8B decoder layers. Exactly those operation positions change from 64 to 110 workers.

Metric	Generic program	110-core specialization	Change
Workers	64	110	+71.9%
Mean kernel time/layer	217.188 μ s	154.932 μ s	-28.66%
Effective BFP8 weight bandwidth	246.2 GB/s	345.2 GB/s	+40.18%
Projected 36-layer saving	—	2.241 ms/token	—

The projected kernel saving is close to the observed 2.147 ms/token. The new program reaches about 92% of the previously measured 375.1 GB/s SwiGLU effective bandwidth. This agreement between operation profile and end-to-end measurement supports the causal attribution.

10.4 Upstream tt-inference-server reference

Tenstorrent’s tt-inference-server is an OpenAI-compatible service built around vLLM and TT-Transformers [25]. It enters the stack above TTNN rather than through JAX, StableHLO, and TT-MLIR:

```

libtt:  SGLang-JAX → JAX/StableHLO → PJRT/TT-XLA/TT-MLIR → TTNN → TT-Metal
TTIS:   OpenAI API → vLLM → TT-Transformers → TTNN → TT-Metal

```

The two stacks use different porting strategies. TTIS uses TT-specific code in the inference engine, transformer implementation, TTNN, and TT-Metal. libtt keeps SGLang-JAX and the JAX model largely unchanged, then optimizes the compiler, runtime, and kernels below PJRT.

Both runs use the same P150, Qwen3-8B model, prompt, 128-token output, serial request policy, disabled prefix cache, and persistent loopback collector. Pure decode uses the 127 intervals from first to last token. End-to-end throughput uses request send to last token. Each run has two warm-ups and 32 retained requests.

Implementation	Pure decode mean $\pm$ SD [95% CI] (tok/s)	Streaming E2E mean $\pm$ SD [95% CI] (tok/s)	TTFT mean $\pm$ SD
libtt current (37d5460)	27.753 $\pm$ 0.330 [27.634, 27.872]	27.619 $\pm$ 0.322 [27.503, 27.735]	58.45 $\pm$ 1.50 ms
TTIS v0.10.0	24.887 $\pm$ 0.532 [24.695, 25.079]	24.776 $\pm$ 0.524 [24.587, 24.964]	63.38 $\pm$ 2.10 ms

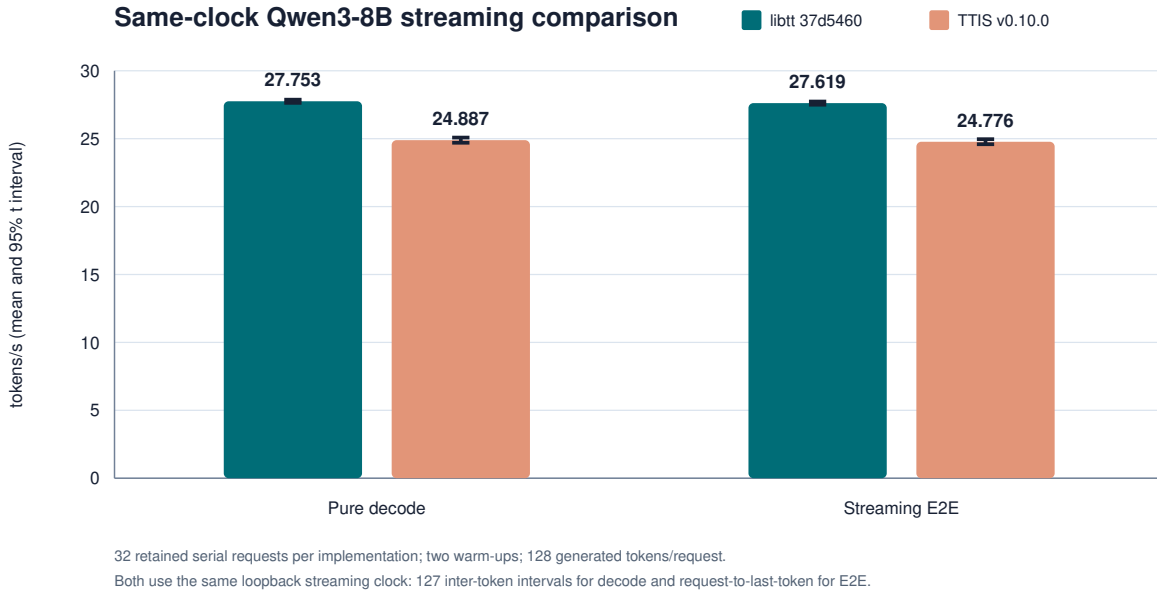


Figure 10.3: External serving-stack reference on the same model workload.

Current libtt is 11.51% faster in pure decode and 11.48% faster end to end. Mean TTFT is 7.78% lower, and mean request-to-last-token latency falls from 5.169 to 4.635 seconds. This compares complete serving stacks, not individual kernels; their model implementations, frontends, and trace strategies differ. The result shows that lower-stack compiler and kernel optimization can match and exceed a stack that also modifies the transformer and inference-engine layers. The tested TTIS artifact is the official v0.10.0 runtime image.

10.5 Correctness and numeric behavior

All 32 requests in each configuration are deterministic. Algebra and reduction changes can alter the token hash. The current kernel hashes are:

Configuration	SHA-256 prefix
SwiGLU block 2, generic down	5119e79e42b5
SwiGLU block 4, generic down	c917933082e3
SwiGLU block 4, 110-core down	c041ccb1901d

Determinism and coherent text do not establish model quality. BF8 arithmetic, reassociation, and reduction order can change logits and greedy choices. Perplexity and task-level evaluation are still required.

11 Engineering rules

11.1 Match the abstraction to the information

Use each level for the information it retains:

- use **TTIR** when the problem is algebraic recognition or graph structure;
- use **TTNN IR** when the compiler needs a named runtime operation, layout, or device dtype;
- use the **TTNN runtime** when the choice depends on concrete tensor and device state;
- use a **TT-Metal program factory** for core topology, circular buffers, multicast, and program selection; and
- use **device kernels** for Dst lifetime, pack/unpack traffic, tile arithmetic, and synchronization.

Lower levels lose graph semantics; higher levels cannot control device data movement.

11.2 Name the removed boundary

The MLP experiments remove different boundaries:

```
shared-LHS matmul:
  removes duplicate activation work, retains doubled output      → -1.69%

SiLU on BinaryNg input:
  removes one elementwise intermediate, retains matmul output    → -0.65%

matmul-SwiGLU epilogue:
  keeps gate/up in Dst and writes only the final half-width result → +2.73%
```

Fusion should name the Dst, L1, DRAM, or launch boundary it removes.

11.3 Down-projection geometry

Profiling identified weight bandwidth as the down-projection limit. Uneven N partitioning uses 110 workers without rereading weights or reducing partials across cores. It reaches 345.2 GB/s with less complexity than a 2D K/N split.

11.4 Keep negative results

Wider SwiGLU blocks reduce partial packs but add CB pressure or reduce overlap. The four-tile result changes throughput by only $+0.27\%$, so two remains the default and the larger values remain experimental. Shared-LHS and consumer-side fusion are kept in the record for the same reason: their measured costs explain the final producer-side design.

12 Limitations and follow-up work

1. **One model and decode regime.** Results apply to Qwen3-8B, a five-token prompt padded to one tile, serial 128-token generation, and a single P150. Other batches, contexts, and scheduling policies may have different limits.
2. **Cumulative attribution is ordered.** The established stages are real cumulative builds, not a factorial experiment. Patch effects can interact.
3. **The foundation group is not decomposed.** Its +22.58% covers several compiler, dtype, validation, and layout changes.
4. **Two metric families.** The older cumulative sequence uses server-reported request latency. The current kernel and TTIS comparison uses the same streaming client clock. The two families are not directly interchangeable.
5. **Sequential sampling.** Configurations were not randomized or interleaved. Thermal and background drift can remain.
6. **Profile naming required positional alignment.** The final profiling CSV lacked operation names. Its 854 rows align with an earlier named trace, and the 36 changed positions match the down projections.
7. **Shape-specific kernels.** The 96-core epilogue and 110-core down projection target Black-hole and exact Qwen3-8B decode shapes.
8. **Quality is unmeasured.** Deterministic coherent completion is a smoke test, not evidence of equivalent perplexity or task accuracy.
9. **External reference scope.** The TTIS v0.10.0 image is official, but P150 model metadata was added locally because that release listed Qwen3-8B on P300. The runtime image was unchanged. Both rows use the same collector but different serving and model stacks.
10. **Training is not measured.** The PJRT/StableHLO boundary can carry training graphs, but operation coverage, collectives, optimizer execution, and numerical quality still need implementation and validation.

The next MLP experiment is to keep the SwiGLU output on chip for the down projection. That requires a shared producer/consumer sharding contract.

13 Conclusion

`libtt.so` keeps the SGLang-JAX and JAX/TPU-facing layers close to upstream while packaging the PJRT backend, compiler, runtime, kernels, and runtime assets in one artifact. One Bazel graph makes the lower stack available for cross-layer optimization.

The sequence starts with a functional backend that runs upstream SGLang-JAX at 16.265 tokens/s. Compiler, runtime, and kernel changes improve end-to-end throughput by 60.61%, and the 110-core down projection reaches 27.753 decode tokens/s. Current libtt is 11.51% faster than the matched TTIS decode mean even though TTIS also uses TT-specific transformer and inference-engine layers.

This report measures inference. The same PJRT/StableHLO architecture can be extended to training without replacing the JAX frontend, once the required operations, collectives, optimizer graphs, and validation are added.

Bibliography

1. Tenstorrent, “TT-XLA.” <https://github.com/tenstorrent/tt-xla>
2. Tenstorrent, “TT-MLIR.” <https://github.com/tenstorrent/tt-mlir>
3. Tenstorrent, “TT-NN Documentation.” <https://docs.tenstorrent.com/tt-metal/latest/ttnn/>
4. Tenstorrent, “TT-Metalium Documentation.” <https://docs.tenstorrent.com/tt-metal/latest/tt-metalium/>
5. Tenstorrent, “TT-UMD: User-Mode Driver for Tenstorrent Hardware.” <https://github.com/tenstorrent/tt-umd>
6. Tenstorrent, “SFPI.” <https://github.com/tenstorrent/sfpi>
7. Z. Tan, B. Kang, and A. Narasimham, “A Developer’s Guide to Debugging JAX on Cloud TPUs,” Google Developers Blog, 2026. Describes `libtpu.so` as the shared library containing the XLA compiler, TPU driver, and hardware communication logic. <https://developers.googleblog.com/a-developers-guide-to-debugging-jax-on-cloud-tpus-essential-tools-and-techniques/>
8. Bazel Project, “Hermeticity.” <https://bazel.build/concepts/hermeticity>
9. OpenXLA Project, “PJRT—Uniform Device API.” <https://openxla.org/xla/pjrt>
10. OpenXLA Project, “StableHLO Specification.” <https://openxla.org/stablehlo/spec>
11. C. Lattner et al., “MLIR: Scaling Compiler Infrastructure for Domain-Specific Computation,” *2021 IEEE/ACM International Symposium on Code Generation and Optimization*, pp. 2–14, 2021. <https://doi.org/10.1109/CGO51591.2021.9370308>
12. Tenstorrent, “TT-Forge: Open-Source AI Compiler Stack.” <https://github.com/tenstorrent/tt-forge>
13. Tenstorrent, “TT-MLIR: Architecture and Dialect Overview.” <https://docs.tenstorrent.com/tt-mlir/overview.html>
14. Tenstorrent, “TT-MLIR Tensor Layout.” <https://docs.tenstorrent.com/tt-mlir/specs/tensor-layout.html>
15. Tenstorrent, “TT-Metalium Getting Started and Programming Model.” https://docs.tenstorrent.com/tt-metal/latest/tt-metalium/get_started/get_started.html
16. Tenstorrent, “Compute Engines and Data Flow within Tensix.” https://docs.tenstorrent.com/tt-metal/latest/tt-metalium/tt_metal/advanced_topics/compute_engines_and_dataflow_within_tensix.html
17. Tenstorrent, “Circular Buffer APIs.” https://docs.tenstorrent.com/tt-metal/latest/tt-metalium/tt_metal/apis/kernel_apis/circular_buffers/circular_buffers.html
18. Tenstorrent, “Tiles.” https://docs.tenstorrent.com/tt-metal/latest/tt-metalium/tt_metal/advanced_topics/tiles.html
19. Tenstorrent, “TTNN Tensor: Layout, Sharding, and BFLOAT8_B.” <https://docs.tenstorrent.com/tt-metal/latest/ttnn/ttnn/tensor.html>
20. Tenstorrent, “Blackhole PCIe Card Documentation” and firmware release notes. https://docs.tenstorrent.com/tt-system-firmware/boards/tenstorrent/tt_blackhole/doc/index.html
21. SGLang Project, “SGLang-JAX.” <https://github.com/sgl-project/sglang-jax>
22. A. Yang et al., “Qwen3 Technical Report,” arXiv:2505.09388, 2025. <https://arxiv.org/abs/2505.09388>

23. B. Zhang and R. Sennrich, “Root Mean Square Layer Normalization,” *Advances in Neural Information Processing Systems 32*, 2019. <https://proceedings.neurips.cc/paper/2019/hash/1e8a19426224ca89e83cef47f1e7f53b-Abstract.html>
24. N. Shazeer, “GLU Variants Improve Transformer,” arXiv:2002.05202, 2020. <https://arxiv.org/abs/2002.05202>
25. Tenstorrent, “tt-inference-server v0.10.0.” <https://github.com/tenstorrent/tt-inference-server/releases/tag/v0.10.0>